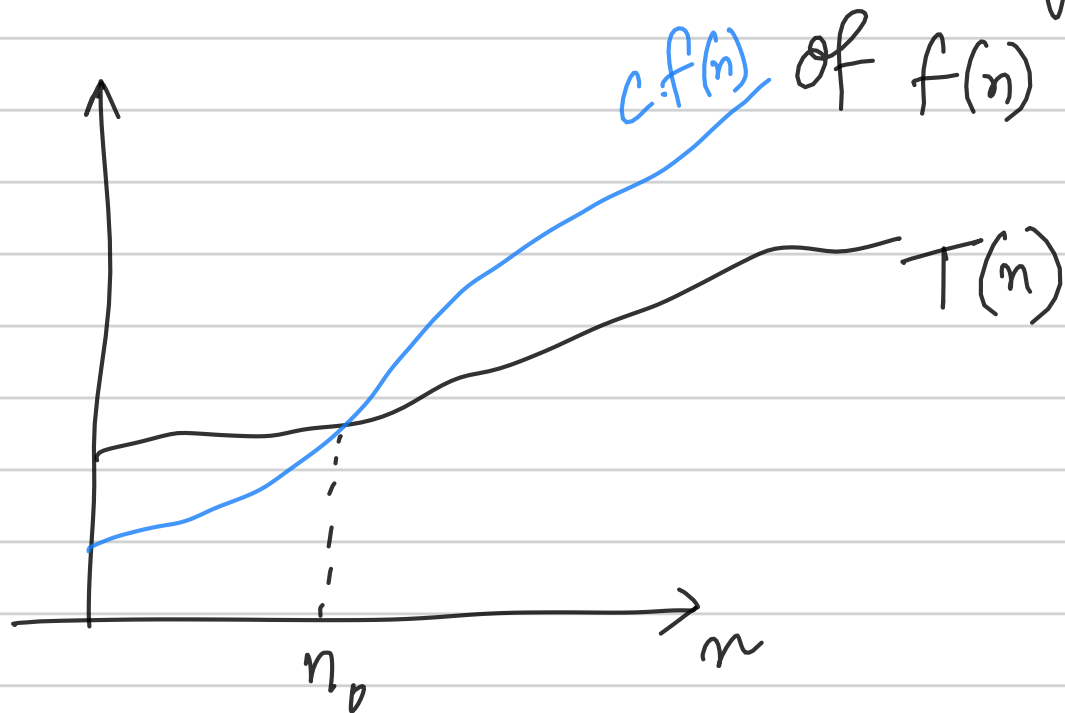


O-notation (Big Oh)

$T(n) = O(f(n))$ iff $T(n)$ is eventually bounded above by a constant multiple of $f(n)$



Math. Defn. $T(n) = O(f(n))$
iff

there exist positive constants c and n_0 such that

$$T(n) \leq c \cdot f(n) \quad \forall n \geq n_0.$$

Eg. ① $T(n) = 5n + 10$ $= O(n)$

$$\leq 5n + n \quad \forall n \geq 10$$

$$= 6n$$

$$\Rightarrow T(n) \leq 6 \cdot n \quad \forall n \geq 10$$

$$c = 6$$

$$n_0 = 10$$

Eg ② $T(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$ $O(n^k)$ $k \geq 0$ and a_i real.

$$\leq |a_k| n^k + |a_{k-1}| n^{k-1} + \dots + |a_1| n + |a_0|$$

$$\leq |a_k| n^k + |a_{k-1}| n^k + \dots + |a_1| n^k + |a_0| n^k$$

$$= \underbrace{(|a_k| + |a_{k-1}| + \dots + |a_0|)}_C n^k \quad \forall n \geq \underbrace{1}_{n_0}$$

Eg ③ $T(n) = n^k$ Is $\underline{T(n) = O(n^{k-1})}$? No

Suppose $T(n) = O(n^{k-1})$

By defn $T(n) \leq c \cdot n^{k-1} \quad \forall n \geq n_0$

$$\Rightarrow n^k \leq c \cdot n^{k-1} \quad \forall n \geq n_0$$

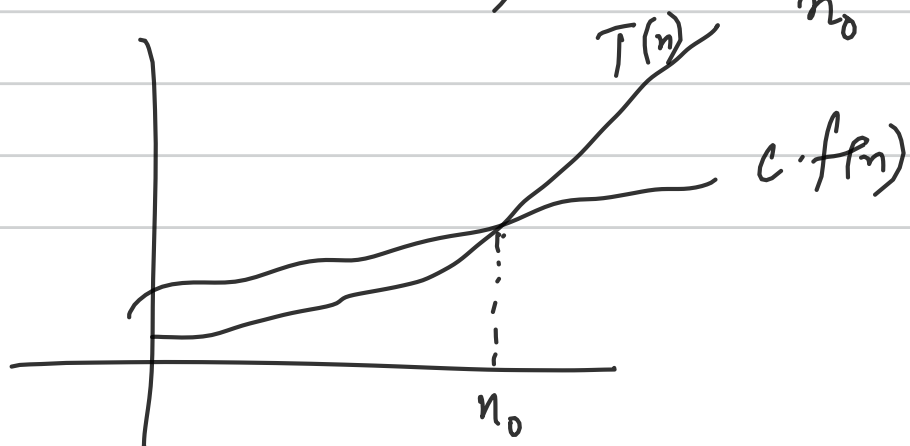
$$\Rightarrow \boxed{n \leq c \text{ for all } n \geq n_0}$$

Contradiction constant.

$\left\{ \begin{array}{l} \text{Eg: } T(n) = 2^{n+10} \\ \text{Eg: } T(n) = 2^{10n} \end{array} \right\}$ Is $T(n) = O(2^n)$.

Big - Omega Notation

$T(n) = \Omega(f(n))$ iff there exist +ve constants c & n_0 s.t. $T(n) \geq c \cdot f(n) \quad \forall n \geq n_0$.



Big Theta Notation.

$$T(n) = \Theta(f(n)) \quad \text{iff} \quad \begin{array}{l} T(n) = O(f(n)) \text{ \& } \\ T(n) = \Omega(f(n)) \end{array}$$

Eg $T(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$

$$T(n) \geq \frac{(a_k - 1)}{2} n^k \quad \forall n \geq \underline{1} \quad \boxed{T(n) = \Omega(n^k)}$$

$$\Rightarrow T(n) = \Theta(n^k)$$

$$T(n) = \Omega(n^2)$$

Divide & Conquer

Merge Sort

$$O(1)$$

Step 1 Divide the array into two roughly equal part; left subarray and right subarray.

$$T\left(\frac{n}{2}\right)$$

Step 2 $C \leftarrow$ Sort the left subarray recursively. (Merge Sort)

$$T\left(\frac{n}{2}\right)$$

Step 3 $D \leftarrow$ Sort the right subarray recursively. (Merge Sort)

$$O(n)$$

Step 4 "Merge" C & D to get the larger sorted array B .

$$T(n)$$

Q. How to merge two sorted arrays C & D to get the larger sorted array B ?

C

2	9	13	16	21
---	---	----	----	----

ⁱ D

26	28	218
----	----	-----

^j

B

2	6	8	9	13	16	18	21
---	---	---	---	----	----	----	----

_k $B[k] = \min\{C[i], D[j]\}$

Size of C & D array are $l/2$

$i = j = 1 = k$

while ($i < l/2$ & $j < l/2$)
 {
 if ($C[i] < D[j]$)
 $B[k] = C[i], i++$
 else
 $B[k] = D[j], j++$

Running time of
merge step = $O(l)$

} $k++$

while ($i < l/2$)
 $B[k] = C[i], i++, k++$

while ($j < l/2$)
 $B[k] = D[j], j++, k++$

Running of Merge Sort:

First we write a recurrence relation for
 the running time.

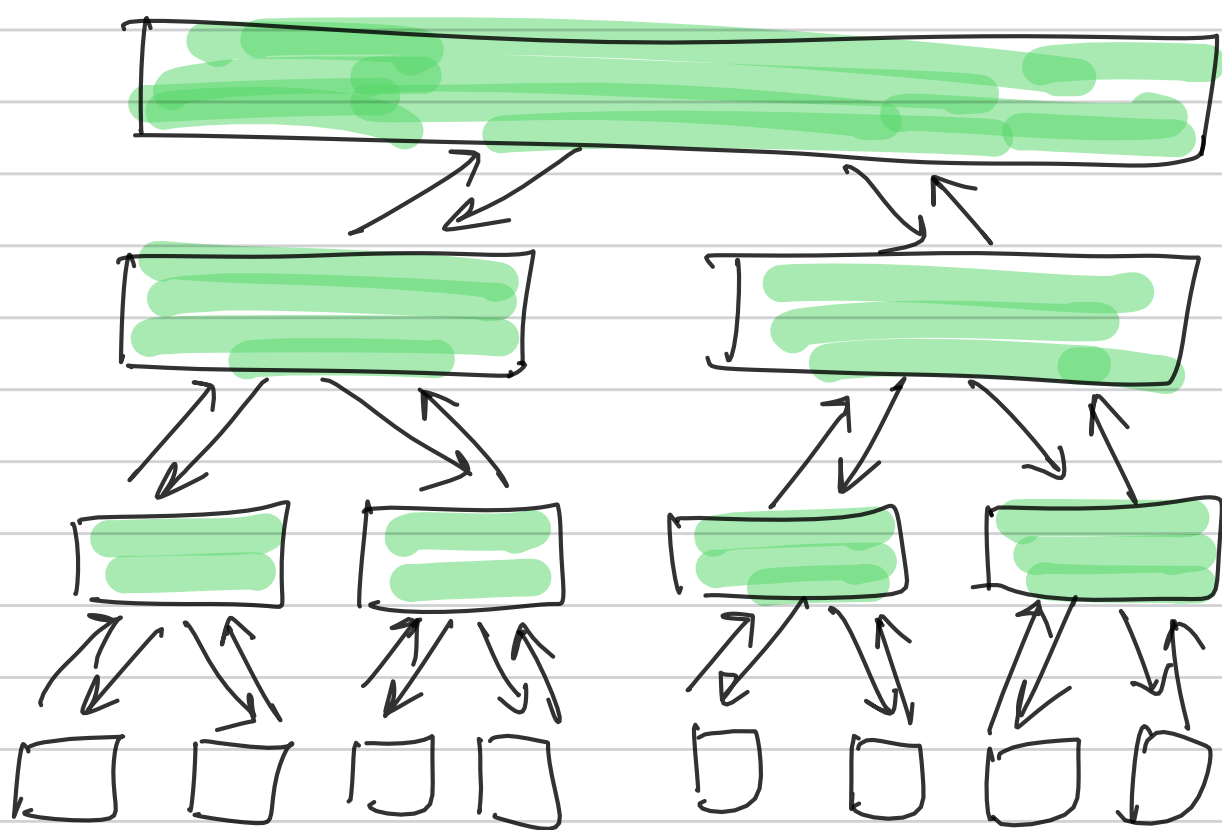
$T(n)$ = Running time for Merge Sort in an array
 of size n .

$$T(n) = O(1) + T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + O(n)$$

$$= 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) \leq \underbrace{2T\left(\frac{n}{2}\right)}_{\text{Recursive work done}} + \underbrace{cn}_{\text{non-recursive work done.}}$$

Recurrence rel for $T(n)$



$$T(n) \leq 2T\left(\frac{n}{2}\right) + C \cdot n$$

$$\leq 2 \left[2 \cdot T\left(\frac{n}{2^2}\right) + C \cdot \frac{n}{2} \right] + Cn$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + Cn + Cn$$

$$\leq 2^2 \left[2 \cdot T\left(\frac{n}{2^3}\right) + C \cdot \frac{n}{2^2} \right] + Cn + Cn$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + Cn + Cn + Cn$$

After k steps

k = depth of recursive merge
Sort to get a single element.

$$T(n) \leq 2^k \cdot \underbrace{T\left(\frac{n}{2^k}\right)}_{c_1} + \underbrace{cn + \dots + cn}_{k \text{ many}}$$

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n.$$

$$T(n) \leq 2^k c_1 + k \cdot cn$$

$$\underline{T(1)} = \text{constant} = c_1$$

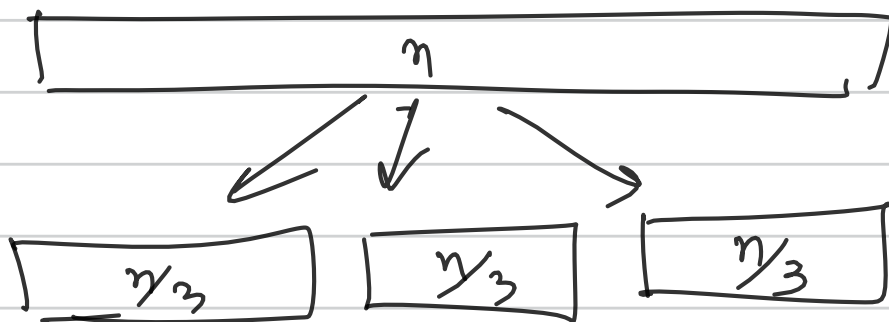
$$= c_1 n + \log_2 n \cdot cn$$

$$= c_1 n + c \cdot n \log_2 n$$

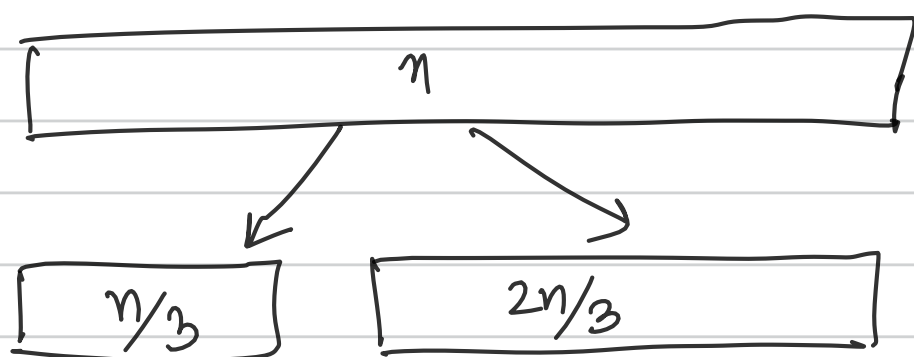
$$= O(n \log_2 n)$$

Ex. Variation of Merge Sort.

a) Divide into 3 equal halves and sort them recursively. Analyse the running time.



b)



Analyse.